Q01 Storage Layer Design · Checkpoint 1

Big Data: Persistence Systems and Data Processing Languages · SDBIS, FEAA, UAIC · Lecturer: Ionuț Hrubaru, PhD

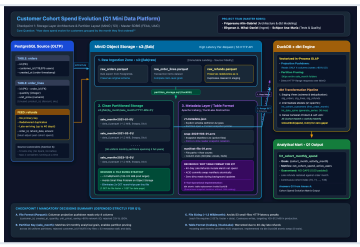
Prigoreanu Alin-Gabriel · Bîrgovan A. Mihai-Daniel · Schipor Ana-Maria

Shop Size: 100M Orders · 3 Years · 4.1 GiB Parquet · DuckDB + MinIO (S3)

Question 1 (Annex A): "How does spend evolve for customers grouped by the month they first ordered?" • **Forces:** A cohort dimension, a complete date spine with no gaps, and atomic snapshot isolation for 40-day late refunds.

1. STORAGE ARCHITECTURE DIAGRAM (MINIO S3 • S3://LAB/)

POSTGRESQL → RAW → BY_MONTH → TABLE FORMAT → DUCKDB



2. PARTITION SIZING ARITHMETIC (COUNT BEFORE WRITE • W5 COMPANION SECTION 6 & 7)

LINES.PARQUET (2,000,001 ROWS • 41.97 MIB) × 100

Candidate Partition Key	Folders (Today)	Rows / File	Today Size (42M)	Folders (100x Shop)	File Size (100x Shop)	Evaluation vs W5 Guidance
month (sale_month = YYYY-MM-01)	36	55,555	1.2 MiB (43.3M total)	36 (fixed 3 yrs)	~120.4 MiB	Optimal (falls in 100–512 MiB production window)
single (no partitioning)	1	2,000,001	41.97 MiB	1	4.10 GiB	Rejected: forces 4.1 GiB full scan on every cohort run
day (sale_day = YYYY-MM-DD)	1,096	1,824	39.2 KiB (59.4M total)	1,096 (fixed 3 yrs)	~3.92 MiB	Rejected: small files multiply listing & footer requests
customer_id	119,975	16	0.4 KiB	~12,000,000 (scales)	0.4 KiB	Catastrophic: millions of tiny files paralyze S3 listings
city	16	125,000	2.69 MiB	16 (fixed)	~268.6 MiB	Rejected: misaligned with monthly cohort analysis
category	6	333,333	7.16 MiB	6 (fixed)	~716.3 MiB	Rejected: too coarse, high data skew between categories

A. File Format (Lands vs Read)

Parquet • Columnar

What lands: Raw Parquet in `s3://lab/raw/` (bulk dump from PostgreSQL). Refunds land as raw CSV/Parquet preserving duplicates as received.

What queries read: Partitioned Parquet in `s3://lab/by_month/`. The two differ in Hive partitioning layout, not columnar encoding.

What queries: Q1 cohort pipeline (`stg_orders`, `stg_lines`, `int_cohort_customers`, `fct_cohort_monthly_spend`).

Rejected & Q1 Reason: CSV / JSON. Cohort spend strictly requires **only 4 columns** (`customer_id`, `created_at`, `quantity`, `unit_price`). Parquet projection pushdown allows DuckDB to fetch only these byte ranges via HTTP Range GETs, saving >80% network I/O.

B. Partition Key & Query Impact

sale_month=YYYY-MM-01

Chosen: `date_trunc('month', created_at)::DATE`. Makes **36 files** of **~120 MiB** at 100x scale (5.55M rows/file).

Speeds up (Q1 Cohorts & Q9 Seasonality): Q1 groups and filters customers by order month. Partition pruning allows DuckDB to skip irrelevant month folders entirely (e.g. Scanning Files: 12/36, zero GETs on skipped files).

Slows down (Q10 LTV & Single Customer): Customer Lifetime Value (Q10) or looking up Customer 4711. Orders are scattered across 36 months; DuckDB must open **all 36 files** (paying 36x footer GET requests) instead of 1 file in a single layout.

C. File Sizing Strategy

Target: 100 – 512 MiB

Target size: Aligned with W5 Section 7 guidance (100 MiB to 512 MiB). For the 100x shop (4.1 GiB over 3 years), 36 monthly files yield **~120.4 MiB per file**, landing squarely in the optimal production window.

Rejected: Day partitioning (1,096 files @ 3.9 MiB) and Customer ID (12M files @ 0.4 KiB), both suffering severe S3 HTTP request round-trip latency.

Handling small writes: When ingestion produces small micro-batches, writes are buffered in memory and coalesced with DuckDB `COPY ... PARTITION_BY`, followed by periodic compaction to consolidate files into target 120 MiB partitions.

D. Table Format Warranted?

Warranted • ACID Snapshots

Can Q1 live with unswitched compaction files? NO. As W5 step 6 proved, compaction leaves old and new files side-by-side; globbing `*.parquet` reads both sets, double-counting rows.

Why Q1 forces a Table Format: Refunds arrive **up to 40 days late**, retroactively modifying past order months' net spend. Rewriting March partitions without an atomic snapshot switch causes dirty reads and duplicated financial metrics.

5-Tool Stack Implementation: To respect the strict 5-tool constraint (no JVM/Iceberg catalog), we implement the table format contract via **dbt atomic table/partition replacement inside DuckDB**, ensuring readers never see uncommitted files.

Annex A Defense ("A cohort dimension and a complete date spine, with no gaps"): Retail customer purchasing is sparse (customers skip consecutive months). An unspined `GROUP BY cohort, order_month` omits inactive months, distorting retention curves and moving averages. In dbt: (1) `int_cohort_customers` locks each customer's `first_order_month`; (2) `int_date_spine` builds a continuous 36-month timeline via `generate_series`; (3) Cartesian cross-join builds all valid (Cohort Month × Activity Month) pairs; (4) `LEFT JOIN` spend with `COALESCE(net_spend, 0.00)` guarantees zero gaps.